



Arquitectura De Computadoras

Fase 1

Proyecto Final

Profesor: Jorge Ernesto Lopez Arce Delgado

José Manuel Alvarez Flores

Héctor Ventura Pérez

Sección D03

Introducción

El procesador MIPS de 32 bits es una arquitectura tipo RISC (Reduced Instruction Set Computer), diseñada para ejecutar instrucciones simples y rápidas con el objetivo de mejorar el rendimiento del procesador. Una de sus principales características es que todas las instrucciones tienen un tamaño fijo de 32 bits. Este procesador trabaja principalmente mediante operaciones entre registros, reduciendo el acceso constante a memoria y aumentando la eficiencia de ejecución.

Elementos y características generales del procesador MIPS de 32 bits

Entre los elementos más importantes del procesador MIPS se encuentra la Unidad Aritmético Lógica (ALU), encargada de realizar operaciones matemáticas y lógicas como sumas, restas, comparaciones y operaciones booleanas AND y OR. Otro componente fundamental es el banco de registros, compuesto por 32 registros de propósito general de 32 bits cada uno. Estos registros permiten almacenar datos temporales, argumentos de funciones, direcciones y resultados de operaciones. Algunos registros tienen funciones específicas, como \$zero, que siempre contiene el valor cero.

El procesador también cuenta con memoria de instrucciones y memoria de datos. La memoria de instrucciones almacena el programa que será ejecutado, mientras que la memoria de datos guarda variables y resultados durante la ejecución. En MIPS se utiliza una arquitectura tipo load/store, lo que significa que únicamente las instrucciones de carga (lw) y almacenamiento (sw) pueden acceder directamente a la memoria; las operaciones aritméticas trabajan solo con datos almacenados en registros.

Otro elemento importante es la unidad de control, la cual interpreta las instrucciones y genera las señales necesarias para coordinar el funcionamiento interno del procesador. Controla el flujo de datos entre registros, memoria y la ALU. Además, el procesador posee un Contador de Programa (PC), que almacena la dirección de la siguiente instrucción a ejecutar. Debido a que cada instrucción ocupa 4 bytes, normalmente el PC aumenta de cuatro en cuatro durante la ejecución secuencial del programa.

Una característica destacada de MIPS es el uso de pipeline, una técnica que divide la ejecución de instrucciones en varias etapas para aumentar el

rendimiento. Generalmente se utilizan cinco etapas: búsqueda de instrucción, decodificación, ejecución, acceso a memoria y escritura de resultados, pueden procesarse simultáneamente en diferentes etapas.

Objetivo

Diseñar un “datapath” con arquitectura tipo MIPS de 32 bits capaz de ejecutar las 28 instrucciones previamente definidas por nosotros.

Objetivos particulares

- Fase 1:
 - Diseñar los módulos necesarios para crear el datapath que permita ejecutar las instrucciones de tipo R de la tabla 1 y tabla 2.
- Fase 2:
 - Agregar los módulos necesarios al datapath para poder ejecutar las instrucciones de tipo I de la tabla 1 y tabla 2.
- Fase 3:
 - Agregar los módulos necesarios para completar el datapath de MIPS de 32 bits para que sea capaz de ejecutar las instrucciones tipo J.
- Fase 4:
 - Crea un algoritmo en código ensamblador, utilizando solo las instrucciones de la tabla 1 y tabla 2.
 - Dicho código debe ser “decodificado” a código binario/maquina, el cual será pre-cargado a la memoria de instrucciones para que el datapath lo ejecute.
 - Crear un programa/script que realice la “decodificación” de su código ensamblador (se recomienda Python).

Desarrollo

Set de instrucciones

Tabla 1

Instrucción	Tipo	Sintaxis
Lw	I	lw \$rt, offset (base)
Sw	I	sw \$rt, offset (base)
slt	R	slt \$rd, \$rs, \$rt
Slti	I	slti \$rt, \$rs, immediate
beq	I	beq \$rs, \$rt, offset
bne	I	bne \$rs, \$rt, offset
j	J	j target
nop	R	nop
bgtz	I	bgtz \$rs, offset

Tabla 2

Instrucción	Tipo	Sintaxis
Add	R	add \$rd, \$rs, \$rt
Sub	R	sub \$rd, \$rs, \$rt
Or	R	or \$rd, \$rs, \$rt
And	R	and \$rd, \$rs, \$rt
Addi	I	addi \$rt, \$rs, immediate
Subi	I	addi \$rt, \$rs, -immediate
Ori	I	ori \$rt, \$rs, immediate
Andi	I	andi \$rt, \$rs, immediate

Con este conjunto de instrucciones se pueden desarrollar muchos algoritmos básicos e intermedios en ensamblador.

Las instrucciones add, sub, mul, addi y subi permiten realizar operaciones matemáticas, lw y sw permiten cargar y guardar datos en memoria, por lo que se puede trabajar con arreglos, matrices y variables almacenadas fuera de los registros, también lt y slti permiten determinar relaciones entre valores, como “menor que”, y sirven para construir decisiones y ciclos junto con beq, bne y bgtz puede implementar estructuras como if, if-else, while, for y do-while. La instrucción j permite realizar saltos incondicionales, útil para repetir bloques de código o cambiar el flujo del programa, nop sirve para insertar ciclos vacíos o controlar riesgos en pipeline y las operaciones lógicas and, andi, or y ori permiten manipular bits, por lo que puede hacer algoritmos relacionados con máscaras binarias, validaciones de banderas, activación o desactivación de bits y operaciones booleanas.

Descripción del desarrollo

Para la fase 1 de este proyecto se realizó un datapath con las instrucciones tipo R de las tablas, para ello se crearon 9 módulos:

- ALU.v.
 - Hace las operaciones tipo R add, sub, and, or, nor, slt y nop.
- ALUctrl.v.
 - Le da instrucciones a la ALU de que operación realizar.
- BR.v.
 - Lee el archivo “bReg.txt” que son los registros en el banco.
 - Escribe y lee registros en el banco.
- DPTR.v.
 - Como tal es el datapath, integra todos los módulos.
- Mem.v.
 - Es la memoria del datapath.
- MemInst.v.
 - Es la memoria en la que se guardan las instrucciones.
 - Lee el archivo “instrucciones.txt” que es donde se almacenan las instrucciones.
 - Lee las instrucciones del archivo y las ejecuta en el datapath.
- MUX2a1.v.
 - Un multiplexor 2 a 1 de 32 bits.
- TB_DPTR.v.
 - Es el testbench del datapath.
- UCtrl.v.
 - Interpreta las instrucciones para generar señales a los distintos módulos del datapath.

Memoria de instrucciones decodificada

El error era un byte faltante, para corregirlo se le agregó el byte al inicio con ceros

```
000000 00001 00000 00100 01010 100000
```

```
add  rs1  rt0  rd4
```

```
000000 00101 00100 10101 00000 100010
```

```
sub  rs5  rt4  rd21
```

```
000000 01000 00100 11000 00000 100100
```

```
and  rs8  rt4  rd24
```

```
000000 01010 01010 11001 00000 100101
```

```
or   rs10 rt10 rd25
```

```
000000 00101 01010 11010 00000 100101
```

```
or   rs5  rt10 rd26
```

```
000000 00000 00000 00000 00000 000000
```

```
nop
```

Propuesta de algoritmo a implementar

El algoritmo a implementar sería la multiplicación matricial ya que requiere de buen control de flujo, manejo de memoria, uso de registros y ciclos anidados. Las multiplicaciones se realizarán a base de sumas repetidas ya que no se cuenta con la instrucción Mul.

Conclusiones

Esta primera fase del proyecto final permitió comprender el funcionamiento de un datapath básico de la arquitectura MIPS con instrucciones de tipo R. Además, se comprendió también cómo se lleva a cabo el proceso de ejecución de instrucciones dentro de un procesador, desde la obtención de la instrucción en binario hasta la generación del resultado final de la ALU. También se observó la importancia de las señales de control y la comunicación entre módulos para el correcto funcionamiento del datapath.

Referencias

Gasparetto, A. (2014). MIPS instruction set. Università Ca' Foscari Venezia.
https://www.dsi.unive.it/~gasparetto/materials/MIPS_Instruction_Set.pdf

MIPS Technologies. (2016). MIPS® architecture for programmers volume II-A: The MIPS32® instruction set manual (Rev. 6.06). MIPS Technologies, Inc.